

Variable Classification Documentation

Step 1: NLSY79 data cleaning pipeline

Katrine Wøldike and Regitze Holt

December 16, 2025

Note: This documentation was created with assistance from Claude 4.5.

Table of Contents

- 1. Executive Summary
- 2. Overview
- 3. Pipeline Execution
- 4. Classification Logic Details
- 5. Embedded Missing Value Detection
- 6. Outlier Detection and Filtering
- 7. Hybrid Variable Detection
- 8. Validation and Corrections
- 9. Output Files
- 10. Verification Results

Executive Summary

This document describes the automated classification of 11,981 variables from the NLSY79 dataset using SAS format mappings, validation, and heuristic analysis. All numbers below reflect the latest pipeline run.

Input: nlsy79_cleaned_4_new.parquet (7,016 × 11,981) **Output:** nlsy79_classified.parquet + variable_classifications.csv (7,016 × 11,981)

Key Pipeline Numbers (latest run)

- **Raw data loaded:** 7,016 rows × 11,981 columns
- **SAS parsing:**
 - 178,014 rename mappings
 - 174,785 format definitions
 - 174,777 format assignments
 - 11,980 SAS variables matched to DataFrame columns
 - 11,980 columns renamed
 - SAS-based classification distribution:
 - continuous: 7,101 (includes 237 year variables + 334 hybrid variables)
 - binary: 3,692
 - categorical_ordinal: 908
 - categorical_nominal: 613
- **Missing value cleaning:**
 - 78 variables with embedded missing codes
 - 2,142 embedded missing code values converted to NA
 - 2 variables skipped (>10% values would be converted)
- **Outlier filtering:**
 - 1,932 STATUS_WK_NUM variables skipped from outlier detection
 - 314,064 outlier values converted to NA in 2,467 variables (3×IQR rule)
 - 0.3736% of total dataset converted (314,064 / 84,058,696 cells)
 - 2,048 variables had outliers skipped due to 1% identical outlier rule
 - Outliers closer to mean than any skipped outlier are also preserved
- **Final classification distribution (after all corrections):**
 - binary: 3,945 (32.9%)
 - continuous: 4,583 (38.3%) - includes 334 hybrid variables
 - categorical_nominal: 2,523 (21.1%)
 - categorical_ordinal: 930 (7.8%)
 - Total variables: 11,981
 - 309 SAS classifications corrected based on data patterns
 - 459 continuous year variables detected (for cleaning), 334 continuous hybrid variables flagged (for tracking)

Hybrid Encoding Detection: - 334 variables detected with mixed categorical codes (0-10) and continuous IDs (100+) - Flagged in output CSV with is_hybrid column for documentation/awareness - Stored as 'continuous' type but marked with is_hybrid=True - Note: These variables are flagged for tracking purposes but are not actively handled or recoded in the pipeline

Overview

Purpose

Accurate variable classification is essential for: - Type-appropriate statistical analysis - Correct imputation strategies - Feature aggregation in longitudinal data - Machine learning model preparation

Classification Schema

Variables are classified into four mutually exclusive types:

Type	Definition	Examples	Count	%
Binary	Exactly 2 unique values	Yes/No, Male/Female	3,945	32.9%
Categorical Ordinal	Ordered categories (3+)	Education level, Likert scales	930	7.8%
Categorical Nominal	Unordered categories	Race, occupation, months	2,523	21.1%
Continuous	Numeric measurements (incl. hybrid)	Income, weight, years, hours	4,583	38.3%

Pipeline Execution

This section documents the classification process in chronological order:

1. **Stage 1:** Load raw data (7,016 × 11,981)
2. **Stage 2:** Parse SAS file (single pass: rename + classify)
3. **Stage 3:** Clean missing values (embedded detection with 10% threshold) **Stage 3b:** Conservative outlier filter (3×IQR with 1% protection rule)
4. **Stage 4:** Classify variables (SAS + heuristics + validation + year conversion)
5. **Stage 5:** Save outputs (classified data + metadata)

Stage 1: Load Raw Data

Input: nlsy79_cleaned_4_new.parquet (7,016 participants × 11,981 variables)

Pre-Cleaning Already Completed:

The input dataset has already undergone extensive pre-cleaning in notebooks: - Negative values converted to NA (-1 to -5 missing codes) - Participant filtering by Rotter Score presence - Variable sparsity filtering: 163,730 variables with >25% NAs removed - Zero-variance filtering: 13 variables removed - Future data removal: 2,259 variables from 2016-2022 removed

What Happens in Step 1:

1. Load the parquet file containing pre-cleaned variables
2. No additional missing value standardization needed (already done)

Result: 7,016 × 11,981 dataset ready for classification.

Note: Participant filtering (>25% missing) is deferred to Step 2 after variable aggregation.

Stage 2: Parse SAS File (Single Pass)

Overview

The SAS file (nlsy79_all_1979-2022.sas , 83.4 MB, ~3.1M lines) contains metadata that translates cryptic reference numbers into meaningful variable names and provides classification hints through format definitions. Instead of reading this massive file multiple times, the parser extracts all needed information in **one pass**.

The Three-Part Metadata Extraction

Part 1: Rename Mappings (178,014 extracted)

What: Maps reference numbers (DataFrame column names) to descriptive variable names

SAS File Syntax:

```
A0003000 = 'VERSION_R30_2022' n
R0000100 = 'CASEID_1979' n
R0173600 = 'SAMPLE_SEX_1979' n
```

Regex Pattern: ([A-Z]\d+)\s*=\s*'([^\s]+)'\n

Result Dictionary:

```
rename_map = {
  'A0003000': 'VERSION_R30_2022',
  'R0000100': 'CASEID_1979',
  'R0173600': 'SAMPLE_SEX_1979',
  # ... 178,014 total mappings
}
```

Purpose: Converts DataFrame columns from R0173600 to SAMPLE_SEX_1979 for readability.

Part 2: Format Definitions (174,785 extracted)

What: Defines how numeric codes map to text labels (the “codebook” for each variable)

SAS File Syntax:

```
value vx0f
  1='Male'
  2='Female';

value vx1f
  0='No'
  1='Yes';

value vx2f
  0='None'
  1='Elementary'
  2='Some HS'
  3='HS Graduate'
  4='Some College'
  5='College Graduate'
  6='Graduate Degree';
```

Regex Patterns: - **Block start:** `value\s+(\w+)` -> Captures format code (e.g., `vx0f`) - **Value mappings:** `(-?\d+(?:-\d+)?|\.\.)='([^\']*')'` -> Extracts `1='Male'` - **Block end:** `;` terminates the format definition

Parsing Logic (State Machine):

```
in_format_block = False
current_format = None
current_mappings = []

for line in sas_file:
    if line matches "value FORMATCODE":
        save previous format if exists
        start new format block
        current_format = "vx0f"
        in_format_block = True

    if in_format_block:
        extract all "NUMBER='LABEL'" from line
        add to current_mappings

    if line contains ';':
        save format_definitions[current_format] = current_mappings
        end format block
```

Result Dictionary:

```
format_definitions = {
  'vx0f': [('1', 'Male'), ('2', 'Female')],
  'vx1f': [('0', 'No'), ('1', 'Yes')],
  'vx2f': [('0', 'None'), ('1', 'Elementary'), ('2', 'Some HS'),
           ('3', 'HS Graduate'), ('4', 'Some College'),
           ('5', 'College Graduate'), ('6', 'Graduate Degree')],
  # ... 174,785 total format definitions
}
```

Purpose: Reveals variable meaning and type - 2 values = binary, ordered values = ordinal, etc.

Part 3: Format Assignments (174,777 extracted)

What: Links variables (by reference number) to their format definitions

SAS File Syntax:

```
format R0173600 vx0f.;
format R0536300 vx1f.;
format R0214800 vx2f.;
```

Regex Pattern: `format\s+([A-Z]\d+)\s+(\w+)\.`

Result Dictionary:

```
format_assignments = {
  'R0173600': 'vx0f', # Sex variable uses Male/Female format
  'R0536300': 'vx1f', # Yes/No variable
  'R0214800': 'vx2f', # Education variable
  # ... 174,777 total assignments
}
```

Purpose: Tells us which format definition applies to each variable.

Connecting the Three Parts

Example 1: Classifying a sex variable

```
Step 1: Look up R0173600 in rename_map
        → 'SAMPLE_SEX_1979'

Step 2: Look up R0173600 in format_assignments
        → 'vx0f'

Step 3: Look up 'vx0f' in format_definitions
        → [('1', 'Male'), ('2', 'Female')]

Step 4: Analyze mappings
        → 2 unique values, unordered categories
        → Classification: BINARY
```

Example 2: Classifying an education variable

```
Step 1: R0214800 → 'HGC_FATHER_1979'
Step 2: R0214800 → 'vx2f'
Step 3: 'vx2f' → [('0', 'None'), ('1', 'Elementary'), ('2', 'Some HS'),
                  ('3', 'HS Graduate'), ('4', 'Some College'),
                  ('5', 'College Graduate'), ('6', 'Graduate Degree')]
Step 4: 7 values with clear ordering (None < Elementary < ... < Graduate)
        → Classification: CATEGORICAL_ORDINAL
```

Example 3: Continuous variable (no format)

```
Step 1: T0889400 → 'TNFI_TRUNC_2012'
Step 2: T0889400 → (no format assignment, or generic numeric format)
Step 3: (no format definition)
Step 4: No format means direct numeric measurement
        → Classification: CONTINUOUS
```

How Classification is Inferred from Value Mappings

After extracting the three dictionaries (`rename_map`, `format_definitions`, `format_assignments`), the function `infer_type_from_sas_format()` analyzes the value mappings to determine variable type.

Classification Logic:

Step 1: Check for Binary (exactly 2 categories)

```

if n_categories == 2:
    # Look for binary label patterns in text
    labels = [label.lower() for _, label in value_mappings]
    all_labels = ' '.join(labels)

    binary_pairs = [
        ('yes', 'no'), ('true', 'false'), ('male', 'female'),
        ('positive', 'negative'), ('present', 'absent')
    ]

    if any pair found in labels:
        return 'binary'

    # Check numeric codes: 0/1 is always binary
    if values == ['0', '1']:
        return 'binary'

    # 1/2 coding is binary for Q-prefixed variables
    if values == ['1', '2'] and (var_name starts with 'Q' or 'item' in name):
        return 'binary'

```

Example:

```

value_mappings = [('1', 'Male'), ('2', 'Female')]
? 2 categories, 'male'/'female' pair found
? Classification: BINARY

```

Step 2: Check for Continuous (>20 categories or ranges)

```

if n_categories > 20:
    # Exception: occupation codes are nominal despite many categories
    if 'occ' in var_name or 'industry' in var_name:
        return 'categorical_nominal'
    return 'continuous'

# Check for range patterns like "100-199"
for value_spec, label in value_mappings:
    if '-' in value_spec: # e.g., "1-999='1 TO 999'"
        return 'continuous'

```

Example:

```

value_mappings = [('1-999', '1 TO 999'), ('1000-1999', '1000 TO 1999'), ...]
? Range pattern detected
? Classification: CONTINUOUS

```

Step 3: Categorical (2-20 categories) - Determine Ordinal vs Nominal

3a. Check for Month Variables (NOMINAL - cyclical)

Months are nominal because they're **cyclical**: December (12) wraps to January (1), making standard ordinal statistics inappropriate.

```

month_keywords = ['january', 'february', 'march', ...]
if any(month in labels):
    return 'categorical_nominal'

# Also check for 1-12 numeric values in month-named variables
if variable matches month pattern and values are 1-12:
    return 'categorical_nominal' # Cyclical data

```

Example:

```

value_mappings = [('1', '1: January'), ('2', '2: February'), ..., ('12', '12: December')]
? Month keywords found
? Classification: CATEGORICAL_NOMINAL (cyclical)

```

3b. Check for Ordinal Patterns

```
# Q-prefixed variables with 3-7 categories = Likert scales
if var_name.startswith('Q') and 3 <= n_categories <= 7:
    return 'categorical_ordinal'

# Look for ordinal keywords in labels
ordinal_patterns = [
    r'strongly (disagree|agree)', # Strongly disagree ... Strongly agree
    r'never.*always',           # Never, Rarely, Sometimes, Often, Always
    r'poor.*excellent',          # Poor, Fair, Good, Very Good, Excellent
    r'none.*all',                # None, Few, Some, Many, All
    r'(low|medium|high)',        # Low, Medium, High
    r'grade|level|degree',       # Education levels
    # ... 16 patterns total
]

if any(pattern_matches(labels):
    return 'categorical_ordinal'
```

Example:

```
value_mappings = [
    ('0', 'None'),
    ('1', 'Elementary'),
    ('2', 'Some HS'),
    ('3', 'HS Graduate'),
    ('4', 'Some College'),
    ('5', 'College Graduate')
]
? 'grade|level|degree' pattern matches
? Clear ordering: None < Elementary < ... < Graduate
? Classification: CATEGORICAL_ORDINAL
```

3c. Default to Nominal

```
# No binary pairs, no ordinal patterns, no special patterns
return 'categorical_nominal'
```

Example:

```
value_mappings = [('1', 'White'), ('2', 'Black'), ('3', 'Hispanic'), ('4', 'Other')]
? No ordinal pattern, no natural ordering
? Classification: CATEGORICAL_NOMINAL
```

Building Final Classifications

After single-pass extraction, the parser matches DataFrame variables to their metadata:

Matching Process:

```

for ref_num, var_name in rename_map.items():
    # ref_num = 'R0173600'
    # var_name = 'SAMPLE_SEX_1979'

    # Check if this reference number is in our DataFrame
    if ref_num not in dataframe_columns:
        continue # Skip variables not in our dataset

    # Get format code for this variable
    format_code = format_assignments.get(ref_num) # 'vx0f'

    # Get value mappings for this format
    value_mappings = format_definitions.get(format_code, [])
    # [('1', 'Male'), ('2', 'Female')]

    # Infer type from mappings
    inferred_type = infer_type_from_sas_format(var_name, value_mappings)
    # Returns: 'binary'

    # Store classification
    classifications[var_name] = {
        'reference_number': ref_num,
        'format_code': format_code,
        'value_mappings': value_mappings,
        'inferred_type': inferred_type
    }

```

Result: - **11,980 variables** successfully matched and classified from SAS metadata - **1 variable** (rotter_score_2) not in SAS file - classified by heuristics - DataFrame columns renamed from reference numbers to descriptive names

Stage 3: Clean Missing Values

Timing: Happens BEFORE classification to avoid misclassifying variables based on missing codes.

Note: Codebook-based missing code conversion was tested but converted 0 values (negative missing codes like -1 to -5 were already cleaned in the pre-processing stage). This step has been removed from the pipeline.

Embedded Missing Value Detection

What Happens:

Detect numeric outliers that are actually missing codes embedded in valid data ranges (e.g., 999 in a 1-10 scale).

Detection Methods (Hybrid Approach):

Method 1: Gap Detection (3× Rule) - If largest value > second_largest × 3, it's likely a missing code - Catches ANY outlier value regardless of specific pattern - Examples: - [1, 2, 3, 4, 5, 999] -> 999 detected (999 > 5×3) - [10, 20, 30, 40, 200] -> 200 detected (200 > 40×3)

Method 2: Common Pattern Matching (Updated) - Checks for: 95, 96, 97, 98, 99, 999, 9999 - For each candidate value: - The next largest value (max_other) is less than the candidate value - The gap between the candidate and max_other is greater than half the candidate value (gap > candidate/2) - This principled gap criterion replaces arbitrary thresholds and makes detection more robust. - Examples: - [1, 2, 3, 4, 95] -> 95 detected (gap 91 > 47.5) - [10, 20, 30, 98] -> 98 detected (gap 68 > 49) - [1, 2, 3, 999] -> 999 detected (gap 996 > 499.5)

Protections: - Skips ID-like variables (values > 100,000) - Skips if >10% of non-null values would be converted - Requires at least 10 values and 2 unique values

Why Hybrid Approach? - Method 1 catches unusual missing codes (e.g., 95, 96, 777) - Method 2 catches standard codes and now also 95–99, using a principled gap - Combination maximizes detection while minimizing false positives

Result: - 78 variables had embedded missing codes detected - 2,142 values converted to np.nan - 2 variables skipped (>10% threshold protection) - Output: step1_embedded_missing_codes.csv (detailed detection report)

Outlier Detection and Filtering

Stage 3b: Conservative Outlier Filter

Timing: After missing code detection, before classification

What Happens:

Apply a conservative 3×IQR outlier filter to numeric variables to detect and convert genuine data errors while preserving valid extreme measurements.

Detection Method:

3×IQR Rule: - Calculate Q1 (25th percentile) and Q3 (75th percentile) - $IQR = Q3 - Q1$ - Flag values as potential outliers if: - Below $Q1 - 3 \times IQR$, or - Above $Q3 + 3 \times IQR$

Selective Conversion with 1% Protection Rule:

Not all flagged outliers are data errors. Some represent legitimate extreme values (e.g., maximum salary caps, benefit limits). The 1% rule protects these:

1. **Identify frequent outliers:** Any outlier value appearing in >1% of detected outliers (not exactly 1%)
2. **Calculate protection threshold:** Minimum distance from mean among frequent outliers
3. **Keep (don't convert to NA):**
 - All frequent outliers (>1% identical)
 - Any outliers closer to the mean than ANY frequent outlier (using closest one)
4. **Convert to NA:** Only "true extreme" outliers farther from mean than all frequent values

Example:

Variable with outliers: [1, 2, 3, 50000, 50000, 50000, 50000, 75000] - Detected outliers by $3 \times IQR$: [50000, 50000, 50000, 50000, 75000] - Frequent outliers (>1%): [50000] ($4/5 = 80\% > 1\%$) - Min distance from mean: $|50000 - \text{mean}|$ - **Keep:** All four 50000 values (frequent) - **Keep:** Any outlier closer to mean than 50000 - **Convert to NA:** 75000 only (farther than closest frequent value)

Rationale:

This approach balances two objectives: - **Remove data errors:** Extreme values that are genuinely mistakes - **Preserve valid extremes:** Repeated extreme values likely represent real phenomena (caps, limits, etc.)

Protections: - Skip variables with <10 non-null values - Skip variables with $IQR = 0$ (no spread)

Result: - Converted 314,064 outlier values to NA in 2,467 variables - 2,048 variables had outliers skipped (>1% identical rule) - Skipped 1,932 STATUS_WK_NUM variables from outlier detection - Preserved frequent extreme values as valid measurements - Output: `step1_outlier_conversions.csv` (detailed conversion report)

Stage 4: Classify Variables

What Happens:

With missing values cleaned, classify all 11,981 variables using a two-tiered approach: 1. **SAS-based classification:** Use format value labels from SAS file (11,980 variables) 2. **Heuristic classification:** Data-driven rules for variables without SAS formats (1 variable: `rotter_score_2`) 3. **Year conversion:** 2-digit years converted to 4-digit during classification (696 year variables detected)

Classification Priority:

1. Special cases (e.g., `rotter_score_2`)
2. SAS format-based -> `infer_type_from_sas_format()`
3. Heuristic rules -> `classify_variable()`
4. Validation -> `validate_sas_classification()`

Year Detection & Conversion:

Years are detected and **converted immediately** during classification (not as a separate step): - **4-digit years (1957-2025):** Already valid, no conversion - **2-digit years (0-99):** Converted based on cutoff: - 0-14 -> 2000-2014 (21st century) - 15-99 -> 1915-1999 (20th century)

Why Convert During Classification?

If a variable is misclassified as continuous instead of year, a separate conversion pass wouldn't catch it. By converting immediately when year pattern is detected, we ensure all year variables are properly handled.

Unified Month Detection:

All classification functions now use a single `is_month_variable()` helper: - Checks patterns: `~m_`, `~m$`, `_m`, `_mo`, `_m$`, `_mo$`, `\bmonth\b` - Prevents month variables from being misclassified as years - Ensures months (1-12) are classified as cyclical nominal, not ordinal

Special Case Handling: ### Manual Review and Hard-coded Categorical Nominal Variables

Some variables with many distinct values, especially occupation and industry codes, were manually reviewed and hard-coded as categorical nominal. This was necessary because, although these variables have many unique codes, they represent discrete, unordered job or industry categories (e.g., 110 = 'Chief executives', 1240 = 'Registered nurses', 5240 = 'Truck drivers') rather than continuous measurements. The affected variables include those with prefixes such as `CPSIND70`, `CPSIND80`, `CPSOCC70`, `CPSOCC80`, `OCCALL-EMP.01`, `EMPLOYER`, `INDUSTRY`, and `OCCUPATION`. This decision was based on both domain knowledge and inspection of variables with the most distinct values, excluding those that were completely unique like `CASEID`. In total, 108 variables were hard-coded as categorical nominal in the pipeline. The full list of these variables is provided in the file `hardcoded_nominal_variables.txt` in the project directory.

1. **STATUS_WK_NUM Variables (1,932 variables)**
 - Problem: Hybrid encoding with 0-9 status codes AND 100+ job identifiers
 - Solution in Step 1: Skipped from outlier detection to preserve valid job IDs
 - Classification: Forced to categorical_nominal (acknowledging hybrid nature)
 - Note: These are NOT recoded in Step 1; recoding happens in Step 2 during aggregation
2. **CPSIND70/CPSIND80 Variables**
 - Problem: Industry codes with hybrid encoding patterns

- Solution: Forced to categorical_nominal (bypasses hybrid detection)
- Reason: Industry codes are categorical despite numeric representation

3. **rotter_score_2** Variable

- Problem: Not in SAS file (consolidated from multiple timepoints)
- Solution: Hard-coded as continuous in SPECIAL_CASES dict

Hybrid Variable Detection:

338 variables detected with mixed encoding: - **Pattern:** Low categorical codes (0-10) mixed with high continuous IDs (100+) - **Detection Logic:** 1. Variable has =10 unique numeric values 2. Has both values <100 AND values =100 3. Low values (<100) are categorical-like (<20 unique, all <100) - **Classification:** Marked as continuous_hybrid -> stored as 'continuous' with is_hybrid=True flag - **Examples:** R_REL-1_1979, R_REL-2_1979, HHI-7_78SCRN - **ML Handling:** May need special treatment (categorical one-hot encoding for low values, continuous for high)

Result: - 11,980 variables classified from SAS formats - 1 variable classified by heuristics (rotter_score_2) - 309 SAS classifications corrected by data validation: - continuous -> binary: 191 - categorical_nominal -> binary: 62 - continuous -> categorical_nominal: 34 - categorical_nominal -> categorical_ordinal: 22 - Additionally detected (for cleaning/tracking purposes only): - 459 year variables (used for 2-digit to 4-digit conversion during cleaning) - 334 hybrid encoding variables (flagged but not handled in pipeline)

Stage 5: Save Outputs

Save classified dataset and classification metadata. See Output Files section for detailed descriptions.

Output Files

The pipeline creates five output files documenting the classification process:

Main Outputs

1. **nlsy79_classified.parquet** (7,016 × 11,981)
 - Classified dataset ready for Step 2
 - All variables renamed and typed
 - Missing values cleaned
 - Years converted to 4-digit format
2. **variable_classifications.csv** (11,981 rows)
 - Classification metadata for each variable
 - Columns: variable_name, variable_type, is_hybrid
 - Used by Step 2 for type-appropriate aggregation

Diagnostic Reports

3. **step1_embedded_missing_codes.csv** (78 rows)
 - Variables with detected embedded missing codes
 - Shows which codes were detected and how many converted
4. **step1_outlier_conversions.csv** (2,467 rows)
 - Variables with outliers converted to NA
 - Shows count and example indices of outliers
5. **step1_hybrid_variables.csv** (334 rows)
 - Variables with hybrid encoding patterns
 - Shows value ranges for low/high categories

Classification Logic Details

This section documents the detailed classification rules used by both SAS-based and heuristic classification.

Why SAS Formats Matter

SAS formats map numeric codes to meaningful labels, revealing variable types:

```

VALUE SEX          # Binary variable
  1 = "Male"
  2 = "Female";

VALUE EDUCATION    # Ordinal variable
  1 = "Less than high school"
  2 = "High school diploma"
  3 = "Some college"
  4 = "Bachelor's degree"
  5 = "Graduate degree";

VALUE INCOME       # Continuous variable
  0 = "0"
  1 = "1 TO 999"
  1000 = "1000 TO 4999"
  5000 = "5000 TO 9999"
  10000 = "10000 TO 999999";

```

Binary Classification

SAS-Based Detection:

```

if len(value_mappings) == 2:
    return 'binary'

```

Heuristic Detection:

```

if len(unique_values) == 2:
    check for patterns: 0/1, yes/no, male/female, true/false
    return 'binary'

```

Examples: - SEX: 1="Male", 2="Female" - EMPLOYED: 0="No", 1="Yes" - MARRIED: 1="Never married", 2="Ever married"

Categorical Ordinal Classification

Detection Logic:

1. Check for ordinal keywords in labels:

```

ordinal_keywords = [
    'strongly', 'very', 'somewhat', 'slightly', # Likert scales
    'excellent', 'very good', 'good', 'fair', 'poor', # Health ratings
    'never', 'rarely', 'sometimes', 'often', 'always', # Frequency
    'none', 'few', 'some', 'many', 'most', # Quantity levels
    'grade', 'level', 'degree' # Education levels
]

has_ordinal_keywords = any(keyword in labels.lower() for keyword in ordinal_keywords)

```

2. Check for ordered numeric ranges in labels:

```

# Patterns like "1-5", "0-10", etc.
range_pattern = r'\d+\s*[-to]\s*\d+'
has_range = any(re.search(range_pattern, label) for label in labels)

```

3. Check category count (3-20 categories):

```

num_categories = len(unique_values)
is_reasonable_ordinal = 3 <= num_categories <= 20

```

Why 3-20 Range? - Fewer than 3: Should be binary - More than 20: Likely continuous, not discrete categories - 3-20: Sweet spot for ordered categories (Likert scales, education levels, etc.)

Examples: - HEALTH_STATUS: 1="Excellent", 2="Very good", 3="Good", 4="Fair", 5="Poor" - EDUCATION: 1="<HS", 2="HS", 3="Some college", 4="Bachelor's", 5="Graduate" - SATISFACTION: 1="Very dissatisfied" ... 5="Very satisfied"

Categorical Nominal Classification

Detection Logic:

1. Check for nominal keywords:

```
nominal_keywords = [
    'race', 'ethnicity', 'hispanic', # Demographics
    'occupation', 'job', 'industry', 'employer', # Employment
    'state', 'region', 'county', 'city', # Geography
    'type', 'kind', 'category', 'class', # Classifications
    'marital status', 'religion' # Personal characteristics
]

has_nominal_keywords = any(keyword in labels.lower() for keyword in nominal_keywords)
```

2. Check for descriptive (non-numeric) labels:

```
# If labels are text descriptions rather than ordered scales
has_descriptive_labels = not all(is_numeric_scale(label) for label in labels)
```

3. Special Case: Months (Cyclical Data)

Problem: Should months be ordinal or nominal?

Traditional approach treats months as ordinal: January < February < ... < December

Why This Is Wrong: - December (12) is not “greater than” January (1) - Distance from December to January should be 1 month, not 11 months - December? January wraps around (cyclical), breaking ordinal assumptions - Computing mean/median of months is meaningless (average of Jan and Dec isn't June)

Our Approach: Classify months as **categorical nominal** (cyclical data)

Detection:

```
# Pattern 1: Variable name contains month indicators
month_indicators = [r'month', r'_m_', r'_mo_', r'stopdate', r'startdate', r'_m$']
is_month_var = any(re.search(pattern, var_name.lower()) for pattern in month_indicators)

# Pattern 2: Values range 1-12
is_month_range = (min(values) == 1 and max(values) == 12)

# Pattern 3: Labels contain month names
month_names = ['january', 'february', 'march', 'april', 'may', 'june',
               'july', 'august', 'september', 'october', 'november', 'december']
has_month_labels = any(month in labels.lower() for month in month_names)

if (is_month_var or has_month_labels) and is_month_range:
    return 'categorical_nominal' # Cyclical, not ordinal
```

Examples: - SCHOOL_ENROLL_M_1979: Month of school enrollment (1-12) - INTERVIEW_DATE_M_2020: Interview month (1-12) - BIRTH_MONTH: Birth month (January-December)

Why Nominal Instead of Other Approaches: - Sine/cosine transformation: $\sin(2\pi \times \text{month}/12)$, $\cos(2\pi \times \text{month}/12)$ - Too complex, less interpretable - Season grouping: Winter/Spring/Summer/Fall - Loses monthly granularity - **Chosen: Categorical nominal** - Simple, correct, preserves cyclical nature without assuming linear order

Examples: - RACE: 1=“Hispanic”, 2=“Black”, 3=“Non-Black/Non-Hispanic” - OCCUPATION: Various occupation types (no inherent ordering) - REGION: Northeast, South, Midwest, West

Continuous Classification

Detection Logic:

1. Check for large numeric range:

```
num_unique = len(unique_values)
is_large_range = num_unique > 20 # More than 20 unique values suggests continuous
```

2. Check for measurement keywords:

```
continuous_keywords = [
    'income', 'wage', 'salary', 'earnings', 'pay', # Financial
    'hours', 'minutes', 'days', 'weeks', 'months', # Time duration
    'weight', 'height', 'bmi', 'pounds', 'inches', # Physical measurements
    'dollars', 'cents', 'amount', 'total', # Monetary
    'number of', 'count', 'how many' # Quantities
]

has_continuous_keywords = any(keyword in var_name.lower() for keyword in continuous_keywords)
```

3. Check for range labels indicating intervals:

```
# Labels like "1 TO 999", "1000 TO 4999" suggest continuous measurement
interval_pattern = r'\d+\s+T0\s+\d+'
has_interval_labels = any(re.search(interval_pattern, label, re.IGNORECASE)
                           for label in labels)
```

Examples: - INCOME_WAGES_SALARY: Dollar amounts (wide range) - HOURS_WORKED_PER_WEEK: Numeric hours (0-80+) - WEIGHT_POUNDS: Body weight measurements

Year Classification

Detection Logic:

1. Check for year keywords:

```
year_keywords = [r'year', r'date', r'birth', r'_y$', r'_yy$']
has_year_keyword = any(re.search(pattern, var_name.lower()) for pattern in year_keywords)
```

2. Check for valid year range:

```
# NLSY79 covers 1957-2022 (birth years + survey years)
is_year_range = (1900 <= min(values) <= max(values) <= 2030)
```

3. Exclude month variables:

```
# Case-insensitive check for month indicators
month_indicators = [r'~m_', r'~m$', r'~m\b', r'_m_', r'_mo_', r'_m$', r'_mo$', r'\bmonth\b']
is_month_var = any(re.search(pattern, var_name.lower(), re.IGNORECASE)
                   for pattern in month_indicators)

if not is_month_var and is_year_range and has_year_keyword:
    return 'year'
```

Why Exclude Month Variables?

Variables like EMPLOYERS_ALL_STARTDATE_ORIGINAL.01~M_XRND contain month values (1-12) but have ~M (month indicator) in the name. Without exclusion, these would be misclassified as years because: - Values 1-12 fall within range 1900-2030 ? - Variable name contains "DATE" ? - But they represent months, not years!

The ~M suffix (case-insensitive matching) identifies month variables and prevents misclassification.

Examples: - SAMPLE_DOB_Y_1979: Birth year (1957-1964) - SURVEY_YEAR: Survey administration year (1979-2022) - DEGREE_YEAR: Year degree completed

Result:

Initial SAS-based classification: - Continuous: 7,101 variables (includes 237 year variables + 334 hybrid variables) - Binary: 3,692 variables - Categorical Ordinal: 908 variables - Categorical Nominal: 613 variables

After validation: - Binary: 3,945 variables (+253 from validation) - Continuous: 4,583 variables (includes 334 hybrid variables flagged for tracking) - Categorical Nominal: 2,523 variables (+1,910 from validation) - Categorical Ordinal: 930 variables (+22 from validation)

Note: Year variables are classified as continuous for ML purposes (696 total year variables: 237 from SAS + 459 from corrections).

Hybrid Variable Detection

What Happens:

Variables with mixed categorical codes (0-10) AND continuous IDs (100+) are detected and flagged.

Detection Logic:

```
if variable_type == 'continuous' and n_unique >= 10:
    low_vals = [v for v in values if v < 100]
    high_vals = [v for v in values if v >= 100]

    if len(low_vals) > 0 and len(high_vals) > 0:
        n_low_unique = len(set(low_vals))
        if n_low_unique < 20 and all(v < 100 for v in low_vals):
            # Flag as hybrid but store as continuous
            flag_as_hybrid(variable)
```

Examples: - R_REL-1_1979: [1, 2, 3, 100, 101, 102, ...] → hybrid - HHI-7_78SCRN: [0-9, 100-999] → hybrid

Handling: - Flagged with is_hybrid=True in output CSV - Stored as 'continuous' type in classifications - Not actively recoded in Step 1 (recoding happens in Step 2)

Result: 334 hybrid variables detected and flagged.

Validation and Corrections

What Happens:

SAS classifications are validated against actual data patterns. When conflicts arise, data reality overrides SAS format assumptions.

Validation Rules

Rule 1: Binary Correction (continuous -> binary)

Trigger: SAS says continuous, but exactly 2 unique values **Logic:**

```
if sas_type == 'continuous' and n_unique == 2:
    if values == [0.0, 1.0]: # Definitely binary
        return 'binary'
    if values == [1.0, 2.0] and (Q-prefixed or 'item' in name):
        return 'binary' # Question/item with 1/2 coding
```

Examples: - SEX: [1, 2] -> binary - EMPLOYED: [0, 1] -> binary - Q05_RESPONSE: [1, 2] -> binary

Rule 2: Year Correction (continuous -> year)

Trigger: SAS says continuous, but values are 4-digit years **Logic:**

```
if sas_type == 'continuous':
    avg_val = mean(values)
    if 1957 <= avg_val <= 2025:
        if all(1900 <= v <= 2100 for v in values[:50]):
            if n_unique <= 30 and all(v >= 1979 for v in values):
                return 'year' # Survey years
```

Examples: - INTERVIEW_YEAR: [1979, 1980, ..., 2022] -> year - BIRTH_YEAR: [1957, 1958, ..., 1964] -> year

Rule 3: Nominal ID Correction (continuous -> categorical_nominal)

Trigger: SAS says continuous, but variable is an ID or occupation code **Logic:**

```
if sas_type == 'continuous':
    # Check for ID patterns
    if 'caseid' in name or 'case_id' in name or name.endswith('_id'):
        return 'categorical_nominal'
    # Check for occupation/industry codes
    if n_unique > 20:
        if 'occ' in name or 'cpsocc' in name or 'industry' in name:
            return 'categorical_nominal'
```

Examples: - CASEID_1979 -> categorical_nominal - OCCUPATION_CODE_2010 -> categorical_nominal - CPSOCC70_1990 -> categorical_nominal

Rule 4: Likert Scale Correction (categorical_nominal -> categorical_ordinal)

Trigger: SAS says nominal, but Q-prefixed with 3-7 categories **Logic:**

```
if sas_type == 'categorical_nominal' and 3 <= n_unique <= 7:
    if name.startswith('q') or name.startswith('Q'):
        return 'categorical_ordinal' # Likely Likert scale
```

Examples: - Q01_SATISFACTION: [1, 2, 3, 4, 5] -> categorical_ordinal - Q12_AGREEMENT: [1, 2, 3, 4, 5, 6, 7] -> categorical_ordinal

Rule 5: Binary from Nominal (categorical_nominal -> binary)

Trigger: SAS says nominal, but only 2 unique values **Logic:**

```
if sas_type == 'categorical_nominal' and n_unique == 2:
    if values == [0.0, 1.0] or values == [1.0, 2.0]:
        return 'binary'
```

Examples: - MARRIED: [0, 1] -> binary - HAS_CHILDREN: [1, 2] -> binary

Rule 6: Hybrid Encoding Detection (continuous -> continuous_hybrid)

Trigger: Numeric variable with both low categorical codes AND high continuous IDs **Logic:**

```

if sas_type == 'continuous' and n_unique >= 10:
    low_vals = [v for v in values if v < 100]
    high_vals = [v for v in values if v >= 100]

    if len(low_vals) > 0 and len(high_vals) > 0:
        n_low_unique = len(set(low_vals))
        if n_low_unique < 20 and all(v < 100 for v in low_vals):
            return 'continuous_hybrid' # Flag for special handling

```

Examples: - R_REL-1_1979: [1, 2, 3, 100, 101, 102, ...] -> continuous_hybrid - STATUS_WK_NUM variables: [0-9, 100-999] -> classified as categorical_nominal (forced, not detected as hybrid)

Validation Result: 309 SAS classifications corrected

Classification Correction Breakdown: - continuous -> binary: 191 - categorical_nominal -> binary: 62 - continuous -> categorical_nominal: 34 - categorical_nominal -> categorical_ordinal: 22

Additional Detection (for cleaning/tracking only): - continuous -> year: 459 (used for 2-digit to 4-digit year conversion during data cleaning) - continuous with hybrid encoding: 334 (flagged with is_hybrid=True but stored as continuous)

Heuristic Classification (No SAS Format)

For variables without SAS formats (1 variable: rotter_score_2), apply data-driven rules:

Heuristic Decision Tree

For each variable:

```

1. Count unique values
+- Exactly 2 -> BINARY
|
+- 3-20 unique values
| +- All numeric, sequential -> Check if ordinal or year
| | +- Range 1900-2030 + year keywords -> YEAR
| | +- Range 1-12 + month indicators -> NOMINAL (cyclical)
| | +- Range 1-31 (days) -> ORDINAL
| | +- Has ordering keywords -> ORDINAL
| | +- Otherwise -> NOMINAL
| |
| +- Non-sequential or text -> NOMINAL
|
+- >20 unique values
+- Range 1900-2030 + year keywords -> YEAR
+- Numeric with measurement keywords -> CONTINUOUS
+- Large categorical set -> NOMINAL

```

Heuristic Implementation Details

Binary Detection:

```

def is_binary(data):
    unique_vals = data.dropna().unique()
    return len(unique_vals) == 2

```

Year Detection (with month exclusion):

```

def is_year_variable(var_name, data):
    # Exclude month variables first (case-insensitive)
    month_indicators = [r'~m_', r'~m$', r'~m\b', r'_m_', r'_mo_', r'_m$', r'_mo$', r'\bmonth\b']
    if any(re.search(pattern, var_name.lower(), re.IGNORECASE) for pattern in month_indicators):
        return False

    # Check for year keywords
    year_keywords = ['year', 'date', 'birth', '_y$', '_yy$']
    has_year_keyword = any(re.search(pattern, var_name.lower()) for pattern in year_keywords)

    # Check for valid year range
    numeric_data = pd.to_numeric(data, errors='coerce').dropna()
    if len(numeric_data) > 0:
        min_val, max_val = numeric_data.min(), numeric_data.max()
        is_year_range = (1900 <= min_val <= max_val <= 2030)

        return has_year_keyword and is_year_range

    return False

```

Month Detection: See Month Detection section for implementation details.

Ordinal vs Nominal Decision:

```
def classify_categorical(var_name, data, num_unique):
    if num_unique == 2:
        return 'binary'

    if num_unique > 20:
        return 'continuous' # Too many for discrete categories

    # Check if values are sequential (1,2,3,4,5 vs 1,3,5,8,12)
    numeric_data = pd.to_numeric(data, errors='coerce').dropna()
    if len(numeric_data) > 0:
        sorted_vals = sorted(numeric_data.unique())
        is_sequential = all(sorted_vals[i+1] - sorted_vals[i] == 1
                             for i in range(len(sorted_vals)-1))

        if is_sequential:
            # Sequential values suggest ordinal (education levels, ratings, etc.)
            return 'categorical_ordinal'

    # Non-sequential or text values -> nominal
    return 'categorical_nominal'
```

Continuous Detection:

```
def is_continuous(var_name, data, num_unique):
    if num_unique <= 20:
        return False # Too few for continuous

    # Check for measurement keywords
    continuous_keywords = ['income', 'wage', 'salary', 'hours', 'weight',
                           'height', 'dollars', 'amount', 'total', 'count']
    has_continuous_keyword = any(kw in var_name.lower() for kw in continuous_keywords)

    # Check if numeric data
    numeric_data = pd.to_numeric(data, errors='coerce').dropna()
    is_mostly_numeric = len(numeric_data) / len(data.dropna()) > 0.9

    return has_continuous_keyword or (is_mostly_numeric and num_unique > 20)
```

Classification Corrections

What Gets Corrected:

The validation stage corrects SAS-based classifications when they conflict with actual data:

- Variables SAS labeled as continuous but have exactly 2 unique values -> Reclassified as binary
- Variables SAS labeled as continuous but are year variables (1957-2025) -> Reclassified as year
- Variables SAS labeled as continuous but are ID codes -> Reclassified as nominal
- Variables SAS labeled as nominal but are Q-prefixed Likert scales -> Reclassified as ordinal
- Variables SAS labeled as nominal but have only 2 values -> Reclassified as binary

Example Correction:

```
Variable: EMPLOYERS_ALL_STARTDATE_ORIGINAL.01~M_XRND
Heuristic Detection: Values 1-12, variable name has "~M" indicator
Classification: categorical_nominal (cyclical month data)
Reasoning: Months wrap around (Dec?Jan), so ordinal statistics are inappropriate
```

Result:

Final classification distribution: - Binary: [TBD] - Continuous: [TBD] - Categorical Nominal: [TBD] - Categorical Ordinal: [TBD] - Year: [TBD]

SAS classifications corrected: [TBD]

Summary Statistics

Missing Value Cleaning Summary

Embedded Missing Code Detection: - Variables with embedded codes: 78 - Values converted to NA: 2,142 - Variables skipped (>10% threshold): 2 - Detection report: `step1_embedded_missing_codes.csv`

Note: Codebook-based cleaning was tested but converted 0 values (negative codes -1 to -5 were already cleaned in pre-processing).

Key Implementation Details

Month vs Year Disambiguation

Problem: Both months and years are numeric, creating potential misclassification.

Solution: Unified `is_month_variable()` helper with case-insensitive matching:

```
def is_month_variable(col_name):
    """Unified month detection used throughout classification."""
    col_lower = col_name.lower()
    month_patterns = [r'~m_', r'~m$', r'~m\b', r'_m_', r'_mo_',
                      r'_m$', r'_mo$', r'\bmonth\b']
    return any(re.search(pattern, col_lower, re.IGNORECASE)
               for pattern in month_patterns)
```

Impact: Prevents month variables from being misclassified as years or having year conversion applied.

Embedded Missing Code Detection

3-Criteria Approach:

Criterion A: Classic gap detection - Gap > 10× median gap AND > 50 - Catches: [1, 2, 3, 4, 5, 999]

Criterion B: Relative outlier detection - Gap > 5× second-largest gap AND > 20 - Catches: [1, 2, 3, 4, 5, 98]

Criterion C: Absolute outlier detection - Gap > 100 AND > 30% of range (excluding suspicious value) - Catches: [10, 20, 30, 40, 9999]

Why Three Criteria?

Different data distributions require different detection strategies. Using multiple criteria increases sensitivity without too many false positives.

Final Outputs

Files Created:

1. **nlsy79_classified.parquet**
 - 7,016 participants × 11,981 variables
 - Renamed columns (descriptive names)
 - Classified variables
 - Missing values cleaned (embedded detection + outlier filtering)
2. **variable_classifications.csv**
 - 11,981 rows
 - Columns: `variable_name`, `variable_type`, `is_hybrid`
 - Hybrid variables flagged with `is_hybrid=True`
3. **step1_embedded_missing_codes.csv** (78 rows)
 - Detailed embedded detection report
 - Columns: `variable`, `detected_codes`, `count_converted`
4. **step1_outlier_conversions.csv** (2,467 rows)
 - Detailed outlier conversion report
 - Columns: `variable`, `outlier_count`, `example_indices`
5. **step1_hybrid_variables.csv** (334 rows)
 - Detailed hybrid variable report
 - Columns: `variable_name`, `min_value`, `max_value`, `count_low_values`, `count_high_values`, etc.

Processing Summary:

- SAS-based classifications: 11,980
- Heuristic-based classifications: 1 (`rotter_score_2`)
- SAS classifications corrected: 309
- Year variables detected for cleaning: 459 (2-digit to 4-digit conversion)
- Hybrid variables flagged: 334 (tracking only)
- Embedded missing values converted: 2,142 (78 variables)
- Outlier values converted: 314,064 (2,467 variables)

Next Steps

Step 2: Trajectory Analysis & ML Preparation

The classified dataset is now ready for Step 2, which includes:

1. **Participant Filtering** (deferred from Step 1)
 - Remove participants with >25% missing data

- Calculate missingness based on final ML-ready features (not raw variables)
 - 2. **Longitudinal Variable Aggregation**
 - Group variables by base name (e.g., INCOME_1979, INCOME_1980, ... -> INCOME)
 - Compute trajectory features (mean, trend, variance, etc.)
 - Apply type-appropriate aggregation methods
 - 3. **Hybrid Variable Handling**
 - Recode inconsistent variables (e.g., STATUS_WK_NUM: 100+ -> 9)
 - Standardize measurement scales across years
 - 4. **Low-Variance Filtering**
 - Remove variables with <1% variance
 - Remove variables where >99% of values are the same
 - 5. **Type-Specific Imputation**
 - Binary: Mode imputation
 - Ordinal/Nominal: Mode or new “missing” category
 - Continuous: Mean/median imputation
 - 6. **Final ML Dataset**
 - Save participant-filtered, aggregated, imputed dataset
 - Ready for machine learning model training
-

Console Output

Expected Runtime: ~2-3 minutes

Actual Execution Output:

```
=====
STEP 1: VARIABLE CLASSIFICATION PIPELINE
=====
```

```
[1.1] Loading raw data and standardizing missing values...
      Loaded: 7,016 rows × 11,981 columns
      Standardized 0 non-standard missing values to np.nan

[1.2] Parsing SAS file for rename mappings and type classifications...
      Reading SAS file (line-by-line)...
          Processed 100,000 lines...
          Processed 200,000 lines...
          ...
          Processed 3,100,000 lines...
      Extracted 178,014 variable name mappings
      Renamed 11,980 columns to descriptive names
      Parsing line-by-line (optimized for large files)...
          Processed 100,000 lines...
          Processed 200,000 lines...
          ...
          Processed 3,100,000 lines...
          Found 178,014 rename mappings
          Found 174,785 format definitions
          Found 174,777 format assignments
      Building classifications for DataFrame variables...
          Matched [TBD] SAS variables to DataFrame columns
      Renamed [TBD] columns to descriptive names
      Successfully classified [TBD] variables from SAS file
      SAS-based classification distribution:
          continuous: [TBD]
          binary: [TBD]
          categorical_ordinal: [TBD]
          categorical_nominal: [TBD]
          year: [TBD]

[1.3] Cleaning missing values (embedded detection)...
      Scanning [TBD] numeric variables...
      Detected embedded missing codes in [TBD] variables
      Converted [TBD] values to np.nan

[1.4] Classifying variables by type...
      Classifying all 11,981 variables...
      Corrected [TBD] SAS classifications based on actual data patterns
      Converted [TBD] 2-digit year variables

      Final classification distribution (year + continuous combined):
          binary: [TBD]
          continuous: [TBD]
          categorical_nominal: [TBD]
          categorical_ordinal: [TBD]

      ---

# Appendix: Example Script Output
```

```
===== STEP 1: VARIABLE CLASSIFICATION PIPELINE =====
```

```
[1.1] Loading raw data and standardizing missing values... Loaded: 7,016 rows × 11,981 columns

[1.1a] Participant filtering deferred to Step 2... Variable sparsity filter (>25% NAs) already applied in pre-cleaning Participant filter will run after variable aggregation in Step 2

[1.2] Parsing SAS file for rename mappings and type classifications... Parsing line-by-line (optimized for large files)... Processed 100,000 lines... Processed 200,000 lines... [... continues to ~3.1M lines ...] Found [TBD] rename mappings Found [TBD] format definitions Found [TBD] format assignments Building classifications for DataFrame variables... Matched [TBD] SAS variables to DataFrame columns Renamed [TBD] columns to descriptive names Successfully classified [TBD] variables from SAS file SAS-based classification distribution: continuous: [TBD] binary: [TBD] categorical_ordinal: [TBD] categorical_nominal: [TBD] year: [TBD]

[1.3] Cleaning missing values (embedded detection)...

[1.3a] Detecting embedded missing codes (999, -9, etc.)... Scanning 11981 numeric variables... Processed 1000/11981 variables... Processed 2000/11981 variables... Processed 3000/11981 variables... Processed 4000/11981 variables... Processed 5000/11981 variables... Processed 6000/11981 variables... Processed 7000/11981 variables... Processed 8000/11981 variables... Processed 9000/11981 variables... Processed 10000/11981 variables... Processed 11000/11981 variables... Detected embedded missing codes in 78 variables Converted 2,142 embedded missing code values to np.nan Skipped 2 variables where >10% of values would be converted (likely valid data) Saved:
step1_embedded_missing_codes.csv Example detections: Q3-3_1979: codes [95.0] (1 values) Q3-4_1979: codes [95.0] (1 values) HGC_LI_1980: codes [95.0] (1 values) HHI_FINAL_GRADECODE.02_1980: codes [95.0] (5 values) HHI_FINAL_GRADECODE.01_1981: codes [95.0] (2 values)
```

[1.4] Classifying variables by type... Classifying all 11981 variables... (Year conversion happens automatically during classification) Corrected 1106 SAS classifications based on actual data patterns

Final classification distribution: binary: 3,692 continuous: 7,101 (includes 237 year variables + 334 hybrid variables) categorical_nominal: 613 categorical_ordinal: 908

?? Detected 338 hybrid encoding variables: (mix categorical status codes with continuous IDs) These will be treated as continuous but may need special handling in ML. First 5: R_REL-1_1979, R_REL-2_1979, HHI-7_78SCRN, HHS-TNFI_TRUNC_78SCRN, C_SAMPWEIGHT_1979 (See variable_classifications.csv for full list)

[1.5] Saving classifications and processed data... Saved: variable_classifications.csv (11981 variables) Saved: nlsy79_classified.parquet (7016 × 11981)

===== CLASSIFICATION COMPLETE
=====

Total variables classified: 11,981 SAS-based: 11,980 Heuristic-based: 1

Next step: Run step2_trajectory_and_ml.py

```
**3. Verify type distribution:**
```python
type_counts = classifications['variable_type'].value_counts()
print(type_counts)

Expected output:
binary 3940
continuous 4583
categorical_nominal 2531
categorical_ordinal 927
```

## Verification Steps

After running step\_1\_classification.py, verify the outputs:

### 1. Check output files exist:

```
ls nlsy79_classified.parquet # Should exist
ls variable_classifications.csv # Should exist
ls step1_embedded_missing_codes.csv # Should exist (embedded detection report)
```

### 2. Verify data dimensions:

```
import pandas as pd

Check classified data
df = pd.read_parquet('nlsy79_classified.parquet')
print(f"Shape: {df.shape}") # Should be (7,016 × 11,981)

Check classifications
classifications = pd.read_csv('variable_classifications.csv')
print(f"Variables classified: {len(classifications)}") # Should be 11,981
```

### 3. Review classification distribution:

```
classifications['variable_type'].value_counts()
```

### 4. Check for unclassified variables:

```
unclassified = classifications[classifications['variable_type'].isna()]
assert len(unclassified) == 0, f"Found {len(unclassified)} unclassified variables"
```

## Verification Results

## Classification Summary

**Final counts after all corrections:** - Binary: 3,945 (32.9%) - Continuous: 4,583 (38.3%) - includes 334 hybrid variables - Categorical Nominal: 2,523 (21.1%) - Categorical Ordinal: 930 (7.8%) - **Total: 11,981 (100%)**

**Corrections applied:** - SAS classifications corrected: 309 - Year variables converted (2-digit → 4-digit): 459 - Hybrid variables flagged: 334

**Data cleaning:** - Embedded missing values: 2,142 (78 variables) - Outlier values: 314,064 (2,467 variables)

# Quality Checks

**All 11,981 variables classified** (no missing types)

**All output files generated**

**Data dimensions preserved** (7,016 × 11,981)

**Variable names readable** (renamed from reference numbers)

## Next Steps

**Run Step 2: Trajectory Analysis & ML Preparation**

```
python step2_trajectory_and_ml.py
```

This will: - Filter participants with >25% missing data (deferred from Step 1) - Handle hybrid variables during aggregation - Aggregate longitudinal variables - Compute trajectory features - Apply type-specific imputation - Prepare final ML-ready dataset

## Appendix: Variable Type Details

### Binary Variables

**Characteristics:** - Exactly 2 unique values - Examples: 0/1, Yes/No, Male/Female, True/False

**Statistical Properties:** - Bernoulli distribution - Mean = proportion of 1s - Variance =  $p(1-p)$  - No scaling needed for ML

**ML Considerations:** - Already in optimal format (0/1 encoding) - Works with all algorithms - Check for class imbalance

### Categorical Ordinal Variables

**Characteristics:** - 3-20 ordered categories - Examples: Education levels, Likert scales, health ratings

**Statistical Properties:** - Ordinal scale (meaningful rank, but distances not necessarily equal) - Median is most appropriate central tendency - Can compute percentiles - Mean is debatable (depends on equal spacing assumption)

**ML Considerations:** - Ordinal encoding preserves order (1, 2, 3, 4, 5) - Can standardize for distance-based models - Decision trees handle naturally without encoding

### Categorical Nominal Variables

**Characteristics:** - Unordered categories (no inherent ranking) - Includes cyclical data (months, days of week) - Examples: Race, occupation, region, months

**Statistical Properties:** - Nominal scale (only equality/inequality meaningful) - Mode is only central tendency - No meaningful distance metric - Chi-square tests for associations

**ML Considerations:** - One-hot encoding for most algorithms (creates binary column per category) - Target encoding for tree ensembles (mean of target per category) - Frequency encoding for high cardinality (count of each category) - For cyclical variables: consider sine/cosine transformation

### Continuous Variables

**Characteristics:** - Numeric measurements on interval/ratio scale - >20 unique values - Examples: Income, weight, hours, age

**Statistical Properties:** - Interval or ratio scale - All summary statistics meaningful (mean, median, std, etc.) - Can compute z-scores and standardize - Distance metrics are meaningful

**ML Considerations:** - Standardization recommended (mean=0, std=1) - Check for outliers (clip or remove) - Check for skewness (log transform if needed) - Consider binning for non-linear relationships

## References

**NLSY79 Documentation:** - Official website: <https://www.nlsinfo.org/content/cohorts/nlsy79> (<https://www.nlsinfo.org/content/cohorts/nlsy79>) - Codebook: <https://www.nlsinfo.org/content/cohorts/nlsy79/topical-guide> (<https://www.nlsinfo.org/content/cohorts/nlsy79/topical-guide>) - User guide: <https://www.nlsinfo.org/investigator/pages/search> (<https://www.nlsinfo.org/investigator/pages/search>)

**Statistical Methods:** - Fisher, N. I. (1993). *Statistical Analysis of Circular Data*. Cambridge University Press. (Cyclical data) - Agresti, A. (2010). *Analysis of Ordinal Categorical Data*. Wiley. (Ordinal variables) - Hoaglin, D. C., Mosteller, F., & Tukey, J. W. (2000). *Understanding Robust and Exploratory Data Analysis*. Wiley.

**Software & Tools:** - Pandas documentation: <https://pandas.pydata.org/docs/> (<https://pandas.pydata.org/docs/>) - SAS format specifications: <https://support.sas.com/documentation/> (<https://support.sas.com/documentation/>) - Python regex reference: <https://docs.python.org/3/library/re.html> (<https://docs.python.org/3/library/re.html>)

